



Kafka 이론(2) - Kafka Streams & Connect

☼ 상태	완료
☰ 다중 선택	

Kafka Streams와 Kafka Connect

출처)

[Kafka Streams와 Kafka Connect](#)

0. 서론

1. Kafka Streams

1-1. KStream

1-2. KTable

1-3. GlobalKTable

1-4. Windowing

1-5. Kafka Streams의 주요 연산

2. Kafka Connect

2-1. Kafka Connect의 주요 구성 요소 5가지

1) Connector

2) Task

3) Worker

4) Converter

5) Transform, SMT(Single Message Transform)

3. CDC(Change Data Capture)

☑ 과제: [Source Connector](#)와 [Sink Connector](#)

1. Source Connector

2. Sink Connector

cf) [Source Connector](#)와 [Sink Connector](#) 차이 정리

☑ 전체 흐름 예시

0. 서론

현대의 데이터 처리 방식은 기존의 **batch(일괄 처리)** 중심에서 **stream(실시간 처리)** 중심으로 확장되고 있음

- 기존 batch 방식은 데이터를 일정 시간 동안 모은 뒤 한 번에 처리하는 방식
- stream 방식은 데이터가 발생하는 흐름을 지속적으로 처리하는 방식

⇒ 데이터를 저장한 뒤 나중에 분석하는 방식만 사용하는 것이 아니라, 데이터가 Kafka에 들어오는 즉시 실시간으로 처리할 수 있음

실시간 데이터 처리를 위한 Kafka의 핵심 기술 2가지 :

- **Kafka Streams : kafka 안에 들어온 데이터를 실시간으로 처리하는 역할**
 - Kafka 토픽의 데이터를 읽어 실시간으로 처리하는 클라이언트 라이브러리
 - Kafka 브로커 내부에서 자동으로 실행되는 기능이 아니라, 개발자가 작성한 애플리케이션 코드에서 실행됨
- **Kafka Connect : 외부 시스템과 kafka 연결을 위한 입구와 출구 역할**
 - Kafka와 외부 시스템을 연결하는 데이터 연동 프레임워크
 - 외부 시스템의 데이터를 Kafka로 가져오거나, Kafka의 데이터를 외부 시스템으로 내보내는 역할

실무에서는 Kafka Connect로 외부 시스템의 데이터를 Kafka에 적재하고,
Kafka Streams로 해당 데이터를 실시간 처리하는 방식처럼 두 기술을 함께 사용하는 경우가 많음

CDC (Change Data Capture) 방식 :

데이터베이스의 변경 사항(insert, update, delete)을 감지해서 kafka로 전달하는 방식

- 기존 방식은 DB를 계속 조회(polling)해야 한다는 문제점이 있었다면,
- CDC 방식은 DB 내부 변경 로그를 읽어서 데이터를 추출한다
 - 변경 발생하는 순간 이벤트 생성 O
 - db 직접 조회 x

1. Kafka Streams

Kafka 토픽의 데이터를 실시간으로 처리하기 위한 클라이언트 라이브러리

(데이터 추상화) 아래의 3가지 핵심 개념을 제공한다 :

1. **KStream** : 발생하는 '이벤트'들 각각을 독립된 데이터로서 기록
예를 들어 이게 이벤트 한개의 데이터 이다. "orderId=1, user=A, amount=10000"
2. **KTable** : 상태의 스냅샷
 - KStream은 무슨 일이 있었는가에 대한 기록이라면,
 - KTable은 해당 시점의 최종적인 상태 = 현재 상태에 집중한다
3. **GlobalKTable** : Kafka Streams의 모든 인스턴스가 전체 데이터를 복제해서 가지고 있음
 - 사용자 정보나 상품 정보 같은 master data를 조인할 때 유용

Windowing과 filter, map, join 등의 연산을 통해 실시간 데이터를 처리할 수 있다 :

1. **Windowing (시간 기반 처리)** : 실시간 데이터는 시간 단위로 묶어야 의미 있는 분석이 가능
 - tumbling window : 고정된 시간 단위 -> 일정한 크기 (1분 단위 매출 합계)
 - hopping window : 시간을 기준으로, 일정 간격으로 이동하며 집계 (최근 5분 데이터를 1분마다 갱신)
 - sliding window : 이벤트 발생 시점을 기준으로 판단 (짧은 시간 내 반복 요청 탐지 -> 이상 패턴)
 - session window : 사용자 활동을 단위로-> 활동이 끊기기 전까지 하나의 세션으로 묶음 (사용자 방문 세션, 앱 사용 분석)5)
2. **kafka streams는 다양한 연산을 통해 데이터를 가공할 수 있음**
ex) filter (특정 조건 데이터만 추출), map (데이터 변환), join (다른 데이터와 결합)

1-1. KStream

KStream은 Kafka 토픽에 들어오는 데이터를 **연속적인 이벤트 흐름**으로 다루는 모델

- 각 이벤트는 독립적인 데이터로 처리된다.
- 예를 들어 아래 데이터는 하나의 주문 이벤트로 볼 수 있다.

```
orderId=1, user=A, amount=10000
```

즉, KStream은 "무슨 일이 발생했는가"에 대한 이벤트 기록(흐름)을 처리하는 데 적합하다.

1-2. KTable

KTable은 **key를 기준으로 현재 상태를 나타내는 테이블 모델**

- KStream이 “무슨 일이 발생했는가”에 대한 이벤트 흐름이라면,
KTable은 “현재 상태가 무엇인가”에 집중한다.
- 예를 들어 같은 userId에 대해 여러 이벤트가 들어오면, KTable은 해당 key의 최신 상태를 기준으로 갱신된다.

즉, KTable은 상태의 스냅샷 또는 최신 상태를 다루는 데 적합하다.

1-3. GlobalKTable

GlobalKTable은 Kafka Streams 애플리케이션의 모든 인스턴스가 **특정 토픽(주제)의 전체 데이터를 복제해서 가지고 있는 테이블 모델**이다.

- 사용자 정보나 상품 정보처럼 크기가 비교적 작고, 여러 파티션의 데이터와 조인되어야 하는 master data를 조인할 때 유용
- 예를 들어
 - 주문 이벤트 토픽에는 **userId**만 들어 있고,
 - 별도의 사용자 정보 토픽에 **사용자 이름**이나 **등급 정보**가 들어있다.
- 이때 GlobalKTable을 사용하면 주문 이벤트와 사용자 정보를 조인해서 처리할 수 있다.

1-4. Windowing

실시간 데이터는 계속해서 들어오기 때문에, 시간 단위로 묶어야 의미 있는 분석이 가능하다

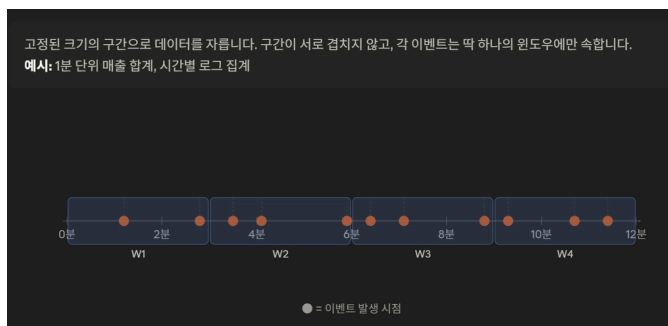
Kafka Streams에서는 Windowing을 통해 특정 시간 구간의 데이터를 묶어서 처리할 수 있다.

• Tumbling Window

- 고정된 시간 단위로 데이터를 묶는 방식
- ex) 1분 단위 매출 집계

```
00:00 ~ 00:01
00:01 ~ 00:02
00:02 ~ 00:03
```

- 각 이벤트는 하나의 window에만 포함된다



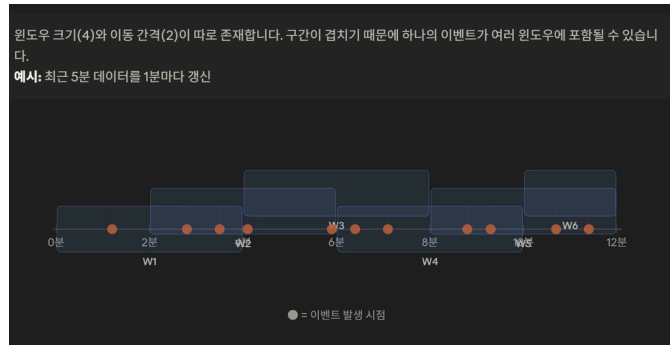
• Hopping Window

- 시간을 기준으로 일정 간격마다 이동하며 집계하는 방식
 - window 크기와 이동 간격이 따로 존재한다. 그래서 구간이 겹칠 수 있다
- ex) 최근 4분 데이터를 2분마다 갱신

```
00:00 ~ 00:04
00:02 ~ 00:06
```

00:04 ~ 00:08

- 하나의 이벤트가 여러 window(4분 단위)에 포함될 수 있다.



- **Sliding Window**

- 이벤트 발생 시점을 기준으로 앞뒤의 일정 시간을 묶는다
 - 그리고 그 시간 안의 이벤트들을 본다
- ex) 짧은 시간 내 반복 요청 탐지, 이상 패턴 탐지

- **Session Window**

- 사용자 활동이 이어지는 동안은 하나의 세션으로 묶고, 일정 시간 동안 활동이 없으면 세션을 종료
 - 활동이 끊기기 전까지 하나의 세션으로 처리
- ex) 사용자 방문 세션, 앱 사용 분석
 - 사용자가 10분 동안 계속 클릭하다가 30분 동안 아무 활동이 없다면, 앞의 10분 활동을 하나의 세션으로 볼 수 있다

1-5. Kafka Streams의 주요 연산

Kafka Streams는 다양한 연산을 통해 데이터를 가공할 수 있다.

- **filter** : 특정 조건을 만족하는 데이터만 추출
- **map** : 데이터 형태를 변환
- **join** : 다른 데이터와 결합

- 예를 들어

```
orders
→ filter(amount > 100000)
→ join(users)
→ map(등급 + 금액 정보 생성)

→ 결과 출력
```

- 주문 이벤트에서 특정 금액 이상인 주문만 필터링하거나, (filter)
- 주문 데이터와 사용자 데이터를 조인하여 (join)
- 사용자 등급별 주문 현황을 분석할 수 있다 (map)

2. Kafka Connect

Kafka Connect는 외부 시스템의 데이터를 Kafka로 가져오거나, Kafka의 데이터를 외부 시스템으로 내보내는 역할

Kafka를 실무에서 사용하면 Kafka 내부의 데이터 처리뿐만 아니라, **외부 시스템과 Kafka를 안정적으로 연결하는 문제도** 중요해진다

- 예를 들어 다음과 같은 작업이 필요할 수 있다

- DB에서 데이터 읽기
- Kafka로 전송
- 실패 시 재시도
- 어디까지 읽었는지 offset 관리
- 병렬 처리
- 장애 복구
- 데이터 포맷 변환

→ Kafka Connect는 이러한 문제를 해결하기 위한 '데이터 연동 프레임워크'

2-1. Kafka Connect의 주요 구성 요소 5가지

1. Connector : 데이터 흐름을 정의하는 객체

- 어디서 데이터를 가져오고, 어디로 보낼지를 정의
 - **Source Connector**: 외부 시스템 → Kafka
 - **Sink Connector**: Kafka → 외부 시스템

2. Task : 실제로 데이터를 옮기는 최소 실행 단위

- 여러 task 로 나누어 병렬 처리

3. Worker : connector와 task가 실행되는 프로세스

- 하나의 worker 또는 여러 worker
 - distributed mode로 여러 워커가 task를 나누어 실행

4. converter : 데이터 포맷 변환

kafka connect 내부 데이터 형식과 kafka 토픽에 저장되는 실제 형식을 다를 수 있음

5. Transform & SMT(Single Message Transform) : 데이터를 옮기는 중간에 간단한 변환을 수행

예를 들어 특정 필드 삭제, 필드 이름 변경, 민감 정보 마스킹, timestamp 추가, topic 이름 변경 등

- 가벼운 비즈니스 로직, 가벼운 전처리 담당
 - 복잡한 비즈니스 로직은 kafka streams에서 하는 것이 맞다

1) Connector

Connector는 **데이터 흐름을 정의하는 객체 (논리적 객체)**

즉, 어디서 데이터를 가져오고 어디로 보낼지를 정의한다

- **Source Connector**
 - 외부 시스템 → Kafka
 - 외부 시스템의 데이터를 Kafka 토픽으로 가져옴
- **Sink Connector**
 - Kafka → 외부 시스템
 - Kafka 토픽의 데이터를 외부 시스템으로 내보냄

2) Task

Task는 실제로 데이터를 옮기는 **최소 실행 단위**

- 하나의 Connector는 여러 Task로 나뉘어 실행될 수 있고
⇒ 이를 통해 병렬 처리가 가능

3) Worker

Worker는 Connector와 Task가 실행되는 **프로세스**

- Kafka Connect는 하나의 Worker로 실행될 수도 있고,
여러 Worker를 사용하는 distributed mode로 실행될 수도 있다.
- distributed mode에서는 여러 Worker가 Task를 나누어 실행
⇒ 확장성과 장애 대응이 좋아

4) Converter

Converter는 **데이터 포맷을 변환하는 역할**

Kafka Connect 내부에서 사용하는 데이터 형식과 Kafka 토픽에 실제로 저장되는 데이터 형식(직렬화 형식)은 다를 수 있다.

- Converter는 다음의 두 형식 사이를 변환
 - Kafka Connect 내부 데이터 형식
 - JSON, Avro, Protobuf 등 실제 저장 형식

5) Transform, SMT(Single Message Transform)

SMT는 데이터를 옮기는 과정에서 **단일 메시지 단위의 간단한 변환**을 수행

- 예를 들어 다음과 같은 작업에 사용할 수 있음
 - 특정 필드 삭제
 - 필드 이름 변경
 - 민감 정보 마스킹
 - timestamp 추가
 - topic 이름 변경
- SMT는 가벼운 전처리에 적합
 - 복잡한 집계, 조인, 상태 기반 처리와 같은 로직은 Kafka Streams에서 처리하는 것이 적절

3. CDC(Change Data Capture)

CDC는 데이터베이스의 변경 사항, 즉 **insert, update, delete**를 감지해서 Kafka로 전달하는 방식

- (문제) 기존 polling 방식은 주기적으로 DB를 조회하여 변경 여부를 확인해야 했다

애플리케이션 → DB 계속 조회 (Polling)

- DB 부하 증가
- 실시간성 떨어짐
- 불필요한 조회 발생
- (해결) 반면 CDC 방식은 DB의 변경 로그를 읽어 변경 사항을 이벤트로 추출

DB → Binary Log → Kafka Connect → Kafka

- 변경이 발생한 순간 바로 이벤트 생성

장점)

- DB를 직접 조회하지 않음 → 부하 감소
- 거의 실시간 데이터 반영
- Insert / Update / Delete 모두 추적 가능

✅ 과제: Source Connector와 Sink Connector

Kafka Connect의 Source Connector와 Sink Connector의 차이를 설명하고, 각각 어떤 실무 상황에서 사용하면 좋은지 예시를 들어 설명하세요.

- Connector : 데이터 흐름 정의하는 객체
 - 어디서 데이터를 가져오고, 어디로 보낼지를 정의

- ① **Source Connector**: 외부 시스템 → Kafka
- ② **Sink Connector**: Kafka → 외부 시스템

- 실무 활용 예시
 - **source connector**가 MySQL(DB, 외부 시스템)의 변경 사항을 감지하고 kafka의 orders 토픽에 데이터를 적재
 - **sink connector**가 특정 토픽의 데이터를 Elasticsearch/관리자 대시보드DB(외부 시스템)으로 보낸다.
 - 관리자는 해당 외부 시스템을 이용해 실시간 이상 결제 알림 등을 확인 할 수 있다.

1. Source Connector

Source Connector는 **외부 시스템의 데이터를 Kafka로 가져오는 Connector**이다.

즉, 데이터 흐름은 다음과 같다.

외부 시스템 → Kafka

예를 들어 주문 서비스의 MySQL DB에서 주문 데이터가 변경된다고 가정한다.

이때 Source Connector는 MySQL DB의 변경 사항을 감지하여 Kafka의 **orders** 토픽에 적재할 수 있다.

MySQL DB → Source Connector → Kafka orders 토픽

이러한 방식은 DB의 변경 데이터를 Kafka로 전달해야 할 때 사용된다.

예를 들어 주문 생성, 결제 상태 변경, 재고 변경 등의 데이터를 Kafka로 보내 실시간 처리의 입력 데이터로 사용할 수 있다.

2. Sink Connector

Sink Connector는 **Kafka의 데이터를 외부 시스템으로 내보내는 Connector**이다.

즉, 데이터 흐름은 다음과 같다.

Kafka → 외부 시스템

예를 들어 Kafka의 **orders** 토픽에 저장된 주문 데이터를 Elasticsearch 또는 관리자 대시보드 DB로 전송할 수 있다.

Kafka orders 토픽 → Sink Connector → Elasticsearch / 관리자 대시보드 DB

이를 통해 관리자는 외부 시스템에 적재된 데이터를 기반으로 실시간 주문 현황이나 이상 결제 알림 등을 확인할 수 있다.

cf) Source Connector와 Sink Connector 차이 정리

구분	Source Connector	Sink Connector
데이터 방향	외부 시스템 → Kafka	Kafka → 외부 시스템
역할	외부 데이터를 Kafka로 가져옴	Kafka 데이터를 외부 시스템으로 보냄
예시	MySQL 변경 데이터를 Kafka 토픽에 적재	Kafka 토픽 데이터를 Elasticsearch에 저장
사용 상황	Kafka로 데이터를 수집해야 할 때	Kafka 데이터를 분석/검색/저장 시스템으로 내보낼 때

✓ 전체 흐름 예시

실무에서는 **Source Connector → Kafka Streams → Sink Connector**가 함께 사용될 수 있다

예를 들어 주문 데이터를 실시간으로 처리하는 구조는 다음과 같다.

```
MySQL DB
↓
Source Connector
↓
Kafka orders 토픽
↓
Kafka Streams
↓
Kafka processed-orders 토픽
↓
Sink Connector
↓
Elasticsearch / 관리자 대시보드 DB
```

이 구조에서 각 구성 요소의 역할은 다음과 같다.

- **Source Connector**
 - MySQL DB의 주문 변경 데이터를 Kafka로 가져옴
- **Kafka Streams**
 - Kafka에 들어온 주문 데이터를 실시간으로 필터링, 변환, 조인, 집계
- **Sink Connector**
 - 처리된 데이터를 Elasticsearch 또는 관리자 대시보드 DB로 전송

이를 통해 주문 데이터의 수집, 실시간 처리, 외부 시스템 적재까지 하나의 데이터 흐름으로 구성할 수 있다.